VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
RICHA SHARMA	**24BCY10067**
Slot	**Date of Submission**
A13	**31ST March 2026**

1. Introduction

Open Source Software (OSS) embodies a paradigm shift in technology. It represents a fundamental change from corporate secrecy to a new paradigm of transparency and collective intelligence. By viewing source code as a public utility rather than a private product, the Open Source movement has democratized innovation.

It has enabled software developers around the world to examine, alter, and redistribute source code. This ensures that the 'building blocks' of our digital world are accessible to everyone, from independent students to global leaders in the technology industry.

The foundation of this new paradigm of collective intelligence is Git. It was built out of necessity. Linus Torvalds developed Git in 2005 after the Linux community was deprived of a proprietary system. It was developed as a Distributed Version Control System (DVCS).

Unlike previous version control systems, where a single entity was required to control a centralized system, Git permits every participant to possess a complete copy of the source code. This is similar to the very essence of the Open Source movement: power to the contributors.

"Protected by the GPLv2 license, Git is a 'copyleft' tool, which means that if you improve it, you must keep it free for the public." This ensures that Git can never be closed off or controlled by corporate forces, which in turn creates a huge ecosystem for platforms like GitHub and GitLab to exist in.

Git, in essence, is the 'universal language' for coding in the future, which enables developers to 'stand on the shoulders of giants' and code the future with free knowledge and freedom.

2. Choosing Your Software

Pickone open-source project from thelist below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Git

3. Project Report Structure

Your report shouldbe12 to 16pageslong.Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

PartA —Origin and Philosophy (Units1&2)

A1. Theproblemthissoftwarewascreatedtosolve

Why Git Was Created?

- In 2005, the development of the Linux kernel came to a halt. They had been using a proprietary system known as BitKeeper, which allowed them to have a distributed system, something other version control systems of the time (such as CVS or SVN) could not handle. When BitKeeper revoked their "free" status for the Linux developers, Linus Torvalds chose to make his own version control system in just two weeks.

The Broken Context

- Before Git, version control was a centralized system, and if something happened to the primary server, development came to a halt. Developers could not work while disconnected from the network, and branching, or creating a new version of a piece of software to test a feature, was a painful and slow process. This was a broken system because it created a bottleneck where one person or one server had to control all of the innovation.

The Decision to Share

- Linus Torvalds created Git as an Open Source project because he understood that "a version control system is only as good as the community that uses it." If it had been closed source, "Git would have been just another version of the BitKeeper problem." By releasing it as Open Source, he made sure that the tool would not be able to be used as leverage against its users.

The Historical Context: A Broken Workflow

- Before 2005, the Linux kernel, which is basically the core of so much tech today, depended on this tool called BitKeeper for managing changes. It was proprietary, and that worked okay for a while, but other options like CVS or Subversion were all centralized setups. Developers had to stay connected to a central server just to commit their work, which meant if the server went down, everything halted. That was pretty frustrating.

- Branching was another hassle, you know, trying out new ideas without messing up the main code. It was tough in those systems, so people avoided it mostly. And there were risks of data getting corrupted too, or things just slowing down a lot.

- Then in 2005, things fell apart with BitKeeper. The free access for Linux folks got pulled, and suddenly this huge project had no way to track its code properly. It felt like a real crisis.
- Linus Torvalds, who started Linux, decided to build something new. He spent about two weeks on the basics of Git, not just to replace BitKeeper but to solve the deeper problems in how teams

collaborate on software. I think he saw how relying on one company's tool could lead to trouble again.

- Instead of keeping Git closed off, he made it open source right away. That way, the whole community owned it, and it wouldn't repeat the same mistake. For a system handling code from around the world, it had to be shared freely.

- Git works as a distributed version control thing. Every person gets a complete copy of the history on their own computer. It puts the control with the developers themselves, not some central spot. Some might say that's a big shift from the old ways, but it seems to make sense for open projects like this. The power moves to the individual, I guess, and that changes how collaboration happens, though it's not always smooth.

A2. Thelicense—whatitactuallysays

Git is licensed under the **GNU General Public License version 2.**

The Four Freedoms

- **Freedom 0:** The freedom to run the program for any purpose.
- **Freedom 1:** The freedom to study how the program works and change it.
- **Freedom 2:** The freedom to redistribute copies.
- **Freedom 3:** The freedom to distribute copies of your modified versions.

Analysis: Git's license provides all these freedoms. However, this license is a "copyleft" license. This means that a company cannot use this license to modify the program, sell it, and keep their modifications secret. They have to share their source code with everyone. This is the "viral" effect of the GPL.

Can company modify and sell it without sharing changes?

No. This is the main point of the GPL. It has a "reciprocity" clause. If a company modifies Git and shares that version, they must make their modified source code available under the same GPLv2 license. They can charge for providing the software, but they cannot make the code proprietary.

GPL vs. MIT: Which is better?

GPL (Protective/Copyleft): This is best for foundational tools like Git or Linux. It ensures that no one can take the code and make it proprietary. It guarantees that the software remains free forever.

MIT (Permissive): This is best for libraries like React, which your project uses. It lets companies use the code in their private projects without needing to share their changes.

My Choice: For your AI Resume Scanner, I would pick the MIT License. Why?
Because it makes it easier for others to adopt. If a startup wants to use your scanner logic in their private hiring platform, the MIT License allows them to do so easily, which helps your code spread more quickly.

'Free as in Beer' vs. 'Free as in Freedom'

- Free Beer (Gratis): This refers to price. You didn't pay to download Git. Free
- Speech (Libre): This focuses on rights.

Example from your project:

- You used the pdfjs-dist library. It was "Free Beer" (cost $0), but more importantly, it was "Free Speech." If the library had a bug that stopped it from reading certain resume formats, you had the freedom to fix the code yourself.

A3. Theethicsofopensource

Should all software be open source?

The Argument FOR: Software runs our lives, from our finances to healthcare systems. If it is not open source, we cannot check for bias, security holes, or "backdoors." Open source promotes equality in education; a student in a small village has access to the same high-quality code as a senior software engineer at Google.

- The Argument AGAINST: Creating software can be very expensive. If a company spends millions developing a specialized medical AI, they may need to keep it proprietary to earn back their investment and feed their programmers. Total open-sourcing could hurt the speed of innovation in a highly competitive commercial environment.

The Ethics of Profiting from Community Labor

Is it ethical for Google or Meta to be utilizing Linux, which was created by volunteers, to make billions of dollars?

- The Reality: It’s a symbiotic relationship. They are making money off it, and at the same time, these organizations are employing thousands of developers to work on these projects during the day and contribute to these projects at night. For instance, Google is one of the biggest contributors to the Linux Kernel and Git.

- The Risk: The ethics come in when a company "leeches" and makes billions off these projects without contributing a single bug fix or dollar back to the community.

Standing on the Shoulders of Giants

This phrase, famously used by Isaac Newton, is the heart of your AI Resume Intelligence project.

- You did not have to write the math for Natural Language Processing from scratch; you stood on the shoulders of the Google Gemini team.

- You did not have to write the logic for rendering a UI; you stood on the shoulders of the React community.

- Does it undermine innovation? No, it enables it. Because you didn't have to spend six months building a "Login Page," you were able to focus your innovation on the AI Matching Algorithm. Open source allows us to stop solving the same problems over and over and start solving new ones.

Part B — Linux Footprint (Unit 2)

1.**Installation Methods**

Git is a foundational tool, so it is available through almost every major Linux package manager. Git is installed via the APT (Advanced Package Tool) package manager. This is easier than "compiling from source" because the community has already pre-packaged the software for

you.

- Setup the Linux Environment using APT
- Open your Linux terminal (or a Virtual Machine/Lab system) and install Git.
- APT(Ubuntu) Code : **sudo apt update && sudo apt install git**

1.**SETTING UP LINUX ENVIRONMENT**

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> wsl --install
Downloading: Windows Subsystem for Linux 2.6.3
Installing: Windows Subsystem for Linux 2.6.3
Windows Subsystem for Linux 2.6.3 has been installed.
The operation completed successfully.
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: anshi
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

2.**INSTALLING GIT USING THE CODE GIVEN**

```
richa_sharma@DESKTOP-5VD9Q3S:~$ sudo apt update && sudo apt install git
[sudo] password for richa_sharma:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
```

```
Get:58 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 c-n-f Metadata [116 B]
Fetched 40.8 MB in 51s (797 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
63 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
```

2.**Key Directories and File Locations**

On a standard Linux installation, Git spreads its components across the filesystem hierarchy to maintain system-wide availability and user-specific customization.

Component	Location	Purpose
Binaries	/usr/bin/git	The main executable file used to run commands.
System Config	/etc/gitconfig	System-wide settings that apply to all users on the

Global Config	~/.gitconfig	User-specific settings (like your name and email).
Local Config	.git/config	Repository-specific settings found inside your project
Internal Logs	.git/logs/	Git does not use /var/log. Instead, it keeps "reflogs"

INSTALLATION OF GIT

```
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$ git --version
git version 2.43.0
```

BINARY LOCATION

```
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$ which git
/usr/bin/git
```

CONFIG DISCOVERY

```
richa_sharma@DESKTOP-5VD9Q3S:~$ ls /etc/gitconfig
ls: cannot access '/etc/gitconfig': No such file or directory
richa_sharma@DESKTOP-5VD9Q3S:~$ ls -a ~
.  ..  .bash_logout  .bashrc  .cache  .gitconfig  .lesshst  .motd_shown  .profile  .sudo_as_a
richa_sharma@DESKTOP-5VD9Q3S:~$ cat ~/ .gitconfig
cat: /home/richa_sharma/: Is a directory
[user]
        name = richa sharma
        email = richas4839@gmail.com
```

3.**User , Group and Security**

- On a standard Linux installation, Git spreads its components across the filesystem hierarchy to maintain system-wide availability and user-specific customization.
- This is a core security feature of Linux. Git can only read or write to files that the current user has permission to access. If you try to git init in a directory owned by root, Linux will block Git.
- This follows the Principle of Least Privilege, ensuring a bug in Git cannot compromise the entire system unless it's being run with sudo.

USER IDENTIFICATION

```
richa_sharma@DESKTOP-5VD9Q3S:~$ whoami
richa_sharma
```

4.**Service Management**

- Unlike a web server, Git is not a persistent service (daemon). It does not stay running in the background.
- Management: It is managed purely through the command line. You verify its "health" simply by checking its version:

VERSION CHECK FOR GIT

```
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$ git --version
git version 2.43.0
```

5.**The Update Model**

- Your Linux system notifies you that updates are available. When you run sudo apt upgrade, your package manager downloads the new Git binary and replaces the old one in /usr/bin/.

UPGRADE FOR GIT

```
richa_sharma@DESKTOP-5VD9Q3S:/mnt/c/WINDOWS/system32$ sudo apt upgrade
[sudo] password for richa_sharma:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
```

PartC—The FOSS Ecosystem (Units3&4)

1. **Dependencies - What Git relies on?**

Git was designed to be fast and portable. Yet, like all programs, Git "stands on the shoulders" of several key open-source libraries and runtimes to implement its network, security, and user interface functionality.

- Compiler: Git is primarily a C program. Thus, Git needs GCC (GNU Compiler Collection) or Clang to build from source code:

Libraries:

- zlib: For high-efficiency data compression (this is why Git repositories are so small!).
- curl: For "Remote" transfer functionality via HTTP/HTTPS (e.g., when you push to GitHub).
- Expat: For parsing XML (used in some older Git network protocols).
- OpenSSL/GnuTLS: To provide secure, encrypted network connections.

Runtimes:

- Tcl/Tk: If you use any of Git's built-in GUI tools like gitk or git-gui, Git needs the Tcl/Tk runtime to draw the GUI windows.
- Perl/Python: Many of Git's auxiliary programs (e.g., git add -i) are actually implemented in Perl or Python.it?

2. **Impact: What Git Has Enabled or Inspired**

Git did not only displace existing technologies; it also gave rise to the "Git as a Service" industry and alternative implementations.

Massive Platforms: GitHub, GitLab, and Bitbucket would not have been conceived without Git.

Alternative Implementations:

- libgit2: A pure C implementation of the core logic of Git, usable as a library to be linked to other programs.
- JGit: A Java implementation of the Git version control system, used as the basis for the

Eclipse IDE. **Inspired Tools:**

- Gitea: A lightweight, self-hosted "Git as a Service" implementation, itself a community fork of the "Gogs" project.
- Gitless: An experimental project from MIT aimed at simplifying the concepts behind Git for new users.

3. **Git and LAMP Stack**

As it is asked where it fits in the LAMP (Linux, Apache, MySQL, PHP) stack.

The Developer's Tool: Git is not in the LAMP acronym, but it is the essential manager for it. While LAMP is the runtime (what the website runs on), Git is the development tool (how the code gets there).

Modern Web Relation: In professional environments, we use a "DevOps" pipeline:

Code is written (PHP/React).

Dimension	Open Source	Proprietary Alternative
Cost	*No licensing fees or per-user costs*	Licensing/Subscription Fees
Security (who can audit the code?)	High Transparency.	Security through Obscurity
Support and reliability	Community-Led	Contracted Support
Freedom to modify	Total Freedom	No Freedom
Community vs corporate	Peer Governance.	Profit Driven
Your overall verdict	Highly Recommended	Optional

Code is committed to Git.

- A "Git Hook" or "Action" automatically deploys that code to the Apache server.
- Given Project: Interestingly, our project uses a MERN-style stack (React, Node, MongoDB). Git is "stack-agnostic"—it works perfectly for your modern AI app just as well as it does for a traditional LAMP site.

4. **Community and Governance**

Git is not owned by a company; it is a community-led project.

Governance: The project is part of the Software Freedom Conservancy (SFC), a non-profit that provides legal and financial help. The "Project Leadership Committee" (Git PLC) makes high-level decisions.

The Maintainer: Since 2005, the "Benevolent Dictator" (Maintainer) has been Junio Hamano. He reviews and approves every major change to the core code.

PartD—OpenSource vs Proprietary (Critical Analysis)

CONCLUSION

After going through the analysis, I feel Git is a solid choice for version control in realworld projects. It's free, easy to work with, and flexible enough to handle both small academic assignments and large-scale software development. One thing I really appreciate is the strong community behind it—there's always support available, and updates keep coming regularly. It also works smoothly across different operating systems, which makes it convenient for teams using varied setups.

I did come across tools like Perforce that handle large files better, but for most development work, Git strikes a good balance between simplicity and costeffectiveness. It's no surprise that it remains the go-to option for many developers and organizations.

Personally, I'd definitely recommend Git, especially for academic projects, startups, or any team working on software. Since it's open source, there's also a chance to look under the hood and maybe even contribute someday. That's something I'd be genuinely interested in—it would help me grow technically while giving back to a tool that so many of us rely on.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Here's a quick overview of what this script does:

- **Purpose** – Generates a system identity report showing key Linux environment details
- **Variables** – Stores information like kernel version, user, uptime, date, and distribution name
- **Command substitution** – Uses $() to fetch real-time data from commands like uname, whoami, and uptime
- **Output formatting** – Uses echo statements to present information in a clean, readable structure
- **Open-source note** – Includes a message about GNU GPL, the license used by most Linux systems

Starter structure

```
GNU nano 7.2                                                    identity.sh
#!/bin/bash

# --- VARIABLES & COMMAND SUBSTITUTION ---
# We store the output of commands into variables using $()
DISTRO=$(lsb_release -ds)
KERNEL=$(uname -r)
CURRENT_USER=$USER
USER_HOME=$HOME
UPTIME=$(uptime -p)
CURRENT_TIME=$(date)

# --- OUTPUT FORMATTING ---
echo "========================================="
echo "      SYSTEM IDENTITY REPORT             "
echo "========================================="
echo "Welcome, $CURRENT_USER!"
echo ""
echo "OS Distribution : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Home Directory  : $USER_HOME"
echo "System Uptime   : $UPTIME"
echo "Current Date/Time: $CURRENT_TIME"
echo ""
echo "-----------------------------------------"
echo "LICENSE NOTICE:"
echo "This operating system and its kernel are"
echo "covered under the GNU General Public License (GPL)."
echo "-----------------------------------------"
```

Output:

```
==========================================
    SYSTEM IDENTITY REPORT
==========================================
Welcome, richa_sharma!

OS Distribution : Ubuntu 24.04.4 LTS
Kernel Version   : 6.6.87.2-microsoft-standard-WSL2
Home Directory   : /home/richa_sharma
System Uptime     : up 45 minutes
Current Date/Time: Mon Mar 30 17:59:18 UTC 2026

LICENSE NOTICE:
This operating system and its kernel are
covered under the GNU General Public License (GPL).
richa_sharma@DESKTOP-5VD9Q3S:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

Let me explain what this script does and how it works behind the scenes.

- **Purpose** – This script checks whether Git (or any specified open-source package) is installed on the system
- **Installation check** – Uses an if-then-else condition along with dpkg and grep to verify if the package is present
- **Displaying details** – If the package is installed, it shows important information
- like the version and description
- **Decision-making** – A case statement is used to print a short explanation about the software based on its name, which demonstrates how conditional branching works in shell scripting
- **What it shows** – Overall, this script combines package management commands with conditional logic to inspect software on a Linux system .

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector

# The software we are auditing
PACKAGE="git"

echo "Checking system for package: $PACKAGE..."

# --- IF-THEN-ELSE LOGIC ---
# dpkg -s checks the status. &>/dev/null hides the technical error message.
if dpkg -s $PACKAGE &>/dev/null; then
    echo "SUCCESS: $PACKAGE is installed."
    echo "----------------------------------------"
    # PIPE WITH GREP: We only want to see the Package name and Version
    dpkg -s $PACKAGE | grep -E 'Package:|Version:|Status:'
else
    echo "FAILURE: $PACKAGE is NOT found."
    echo "Install it using: sudo apt install $PACKAGE"
fi

echo "----------------------------------------"

# --- CASE STATEMENT ---
# This prints a specific philosophy note based on the package name
case $PACKAGE in
    git)
        echo "Philosophy: The distributed system that democratized code collaboration."
        ;;
    apache2)
        echo "Philosophy: The web server that built the open internet."
        ;;
    mysql-server)
        echo "Philosophy: Open source at the heart of millions of web applications."
```

```
    mysql-server)
        echo "Philosophy: Open source at the heart of millions of web applications."
        ;;
    firefox)
        echo "Philosophy: The browser that fights for a free and open web."
        ;;
    vlc)
        echo "Philosophy: The universal player that respects user freedom."
        ;;
    *)
        echo "Philosophy: A community-driven tool for the modern digital era."
        ;;
esac
echo "----------------------------------------"
```

Output:

```
Checking system for package: git...
SUCCESS: git is installed.
---
Package: git
Status: install ok installed
Version: 1:2.43.0-1ubuntu7.3
---
Philosophy: The distributed system that democratized code collaboration.
richa_sharma@DESKTOP-5VD9Q3S:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

Let me walk you through what this script does and how it puts together several shell scripting concepts.

- **Purpose** – This script audits important system directories by looping through a predefined list
- **Looping** – Uses a for loop to go through each directory in the list one by one
- **Directory checks** – For each directory, it checks if it exists, then retrieves permissions, owner, and size using commands like ls -ld and du -sh
- **Text processing** – Uses tools like awk and cut to extract specific fields from command outputs
- **Git config check** – Additionally, it checks for the presence of the Git configuration file in the user's home directory and displays its details
- **What it demonstrates** – This script brings together looping, conditionalchecking, and text processing—all essential concepts in shell scripting

Starter structure

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

# --- THE LIST (ARRAY) ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "       DIRECTORY AUDIT REPORT           "
echo "========================================"
echo "DIRECTORY        | PERMS/OWNER      | SIZE"
echo "----------------------------------------"

# --- THE FOR LOOP ---
# This loops through each path in the DIRS array
for DIR in "${DIRS[@]}"; do
    # Check if the directory actually exists
    if [ -d "$DIR" ]; then
        # Use ls -ld to get info, then AWK to extract:
        # $1 (Permissions), $3 (Owner), $4 (Group)
        PERMS=$(ls -ld "$DIR" | awk '{print $1 " (" $3 ":" $4 ")"}')

        # Use du (Disk Usage) -s (summary) -h (human readable)
        # 2>/dev/null hides 'permission denied' errors for certain subfolders
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        # Output the result in a formatted line
        printf "%-16s | %-16s | %s\n" "$DIR" "$PERMS" "$SIZE"
    else
        echo "$DIR => Error: Directory not found."
    fi
done
echo "========================================"
```

Output:

```
========================================
    DIRECTORY AUDIT REPORT
========================================
DIRECTORY | PERMS/OWNER | SIZE
========================================
/etc              | drwxr-xr-x(root:root) | 4.3M
/var/log          | drwxrwxr-x(root:syslog) | 17M
/home             | drwxr-xr-x(root:root) | 32K
/usr/bin          | drwxr-xr-x(root:root) | 95M
/tmp              | drwxrwxrwt(root:root) | 24K
========================================
richa_sharma@DESKTOP-5VD9Q3S:~$
```

Script 4 — Log File Analyzer (Units 2 & 5)

Let me break down what this script does and how it handles log file analysis.

- **Purpose** – This script analyzes a log file to count how many times a specific keyword appears
- **Command-line input** – Takes input using arguments ($1 and $2) so the user can specify the file and keyword when running the script
- **Line-by-line processing** – Uses a while read loop to go through the file line by line
- **Keyword matching** – An if condition with grep checks whether each line contains the specified keyword
- **Counting matches** – A counter variable keeps track of how many matches are found.
- **Error handling** – Handles errors like missing files or empty files to make the script more robust.
- **Displaying results** – Shows the last five matching lines using tail for quick reference.
- **What it demonstrates** – This script brings together file handling, loops, conditional logic, and error checking in a practical way

Starter structure

```
# --- WHILE-READ LOOP ---
# Reads the file line by line
while IFS= read -r LINE; do
    # -i makes it case-insensitive, -q makes it quiet
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# --- SUMMARY & TAIL ---
echo "Scan Complete."
echo "Keyword '$KEYWORD' found $COUNT times."
echo ""
echo "Last 5 matching entries:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
echo "----------------------------------------"
```

Output:

```
Last 5 matching entries:
2026-03-30T17:13:41.441805+00:00 DESKTOP-5VD9Q3S systemd[1]: apport-autorepo
rt.path - Process error reports when automatic reporting is enabled (file wa
tch) was skipped because of an unmet condition check (ConditionPathExists=/v
ar/lib/apport/autoreport).
2026-03-30T17:13:41.441811+00:00 DESKTOP-5VD9Q3S systemd[1]: apport-autorepo
rt.timer - Process error reports when automatic reporting is enabled (timer
based) was skipped because of an unmet condition check (ConditionPathExists=
/var/lib/apport/autoreport).
2026-03-30T17:13:41.668205+00:00 DESKTOP-5VD9Q3S snapd[200]: helpers.go:190:
 error trying to compare the snap system key: system-key missing on disk
2026-03-30T17:13:43.979281+00:00 DESKTOP-5VD9Q3S wsl-pro-service[204]: #033[
33mWARNING#033[0m Could not ensure valid Landscape configuration: could not
ensure valid Landscape configuration: could not register distro to Landscape
: could not enable Landscape: /usr/bin/landscape-config: error: exit status
1.
2026-03-30T17:24:47.700467+00:00 DESKTOP-5VD9Q3S wsl-pro-service[204]: #033[
33mWARNING#033[0m Exiting after could not ensure valid Landscape configurati
on: could not register distro to Landscape: could not enable Landscape: /usr
/bin/landscape-config: error: exit status 1.

richa_sharma@DESKTOP-5VD9Q3S:~$
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

Let me explain how this script creates a personalized manifesto through user interaction.

- **Purpose –** This script interactively generates a personalized open-source manifesto.
- **Collecting input –** Uses the read command to take input from the user and stores their responses in variables.
- **Building content –** Combines these inputs into a meaningful paragraph using string concatenation.
- **Adding timestamp –** Uses the date command to include the current date in the manifesto.
- **Saving to file –** Writes the output to a text file using redirection (> and >>).
- **Displaying output –** Finally, shows the generated manifesto on the screen.
- **What it demonstrates –** This script brings together user interaction, file handling, and dynamic content creation in a simple but effective way.

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

# CONCEPT DEMONSTRATION: Aliases
# In a real shell, you could add: alias myman='cat ~/manifesto_$(whoami).txt'
# to your .bashrc to quickly view this file anytime with one command.

echo "========================================"
echo "    THE OPEN SOURCE MANIFESTO GENERATOR   "
echo "========================================"
echo "Answer three questions to define your philosophy."
echo ""

# --- INTERACTIVE USER INPUT ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- DATA PREPARATION ---
DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)
OUTPUT="manifesto_${USER_NAME}.txt"

# --- COMPOSING THE MANIFESTO (String Concatenation & Redirection) ---
# The '>' symbol creates a new file (overwriting old ones)
echo "OPEN SOURCE MANIFESTO" > "$OUTPUT"
echo "Generated on: $DATE" >> "$OUTPUT"
echo "----------------------------------------" >> "$OUTPUT"

# Composing the paragraph
MANIFESTO_TEXT="As a developer and believer in open collaboration, I recognize that tools like $TOOL are the foundation of our digit>

# The '>>' symbol appends the paragraph to the file
```

```
# Composing the paragraph
MANIFESTO_TEXT="As a developer and believer in open collaboration, I recognize that tools like $TOOL are the foundation of our digit>

# The '>>' symbol appends the paragraph to the file
echo "$MANIFESTO_TEXT" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Signed, $USER_NAME" >> "$OUTPUT"

echo ""
echo "Success! Your manifesto has been saved to $OUTPUT"
echo "----------------------------------------"
# --- PRINTING THE RESULT ---
cat "$OUTPUT"
echo "----------------------------------------"
```

Output:

```
==========================================
 THE OPEN SOURCE MANIFESTO GENERATOR
==========================================
Answer three questions to define your philosophy.


1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? independence
3. Name one thing you would build and share freely: website
-bash: !': event not found

Success! Your manifesto has been saved to manifesto_richa_sharma.txt
---
PRINTING THE RESULT ---
OPEN SOURCE MANIFESTO
Generated on: 30 Mar 2026
---
As a developer and believer in open collaboration, I recognize that tools li
ke git are the foundation of our digital world. Freedom, to me, means indepe
ndence. I pledge to build website and share it freely with the community.
---
Signed, richa_sharma

richa_sharma@DESKTOP-5VD9Q3S:~$
```

7. Reference Rosources

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html •Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.